ChatGPT

Executive Summary

The semiconductor landscape has evolved rapidly in the past five years, driven by AI demand and Moore's Law slowing. Advanced transistors (FinFET successors like gate-all-around [GAA] nanosheets and novel channel materials) promise continued scaling【5†L32-L39】【41†L1-L4】. Packaging technologies (chiplets, 2.5D/3D stacking, HBM, UCIe) are enabling modular, high-bandwidth designs【13†L89-L97】【8†L35-L43】. Memory innovations (high-bandwidth HBM, analog/digital compute-in-memory [CIM]) are critical for AI throughput【13†L89-L97】【12†L53-L61】. On-chip interconnects still use 2D/3D mesh fabrics, with emerging interest in silicon photonics for future high-speed links. The open-source EDA ecosystem (Yosys, OpenROAD, OpenSTA, Magic, etc.) has matured, offering accessible flows for RTL synthesis, placement/routing, static timing, and layout signoff【30†L105-L114】【30†L129-L136】. Manufacturing has seen mass use of EUV lithography (pushing sub-7nm nodes) and sophisticated yield modeling (Poisson, Murphy, Seeds, etc.) to predict silicon cost【5†L105-L113】【35†L1-L4】.

Key insights: GAAFETs resolve FinFET limits in sub-3nm logic【5†L32-L39】; HBM is now ubiquitous in AI ASICs【13†L89-L97】; CIM (especially analog and hybrid) offers orders-of-magnitude energy savings for certain workloads【12†L53-L61】; UCIe 3.0 is establishing open chiplet standards【8†L35-L43】; open-source tools like Yosys/OpenROAD are increasingly production-capable【30†L105-L114】; and realistic wafer-yield models (Poisson, Murphy) are essential to cost estimates【35†L1-L4】.

To exploit these, we map *Claude-driven* prompt engineering projects to SilicaFlux development. These include designing synthesizable RTL (SRAM wrappers, DMA engines, accelerators), Python simulation modules (memory roofline, power/IR solvers, thermal models, yield simulators), and EDA pipelines (OpenROAD floorplanning, DFT flows, FPGA build scripts). Each project is specified with objectives, inputs/outputs, tools, deliverables, metrics, and risk/effort. We prioritise high-impact, implementable tasks (e.g. SRAM wrappers, Roofline simulator, MBIST, power-grid solver, memory event simulator, FPGA flow, wafer-yield model) and compare them in terms of expected PPA improvement vs. development effort.

A staged roadmap is proposed to integrate Claude-generated code into SilicaFlux's repository and TwinLab platform. It defines milestones (baseline SilicaFlux-V prototype, memory subsystem integration, floorplan optimisations, DFT integration, performance signoff), timeline (0–12 months), CI and data-traceability practices, and success criteria at each gate. We provide *actionable Claude prompt templates* for at least eight key tasks (e.g. "Implement parameterised SRAM wrapper"), describing repository introspection steps, expected outputs (RTL, scripts, reports), and acceptance tests (simulation success, lint, synthesis, formal checks).

Ethical/IP/Security: Using LLMs to generate semiconductor IP raises concerns: IP leakage (training data biases), code provenance, compliance with IP licenses, and security of design secrets. We discuss safeguards (private model deployment, code review, licensing checks, audit logs) and governance policies (human oversight, restricted data use, contributor agreements).

Visual aids include a mermaid Gantt chart of the development timeline, an ER diagram of the design ecosystem (Claude agents, repo, EDA tools, TwinLab UI, CI, PDK libraries), and tables comparing projects by impact vs. effort. In summary, this deep research identifies state-of-the-art trends and maps

them to specific Claude-driven engineering tasks, providing a concrete plan for SilicaFlux's evolution from RTL to pre-fabrication maturity.

1. Technology Survey and Trends

1.1 Transistor Advances (FinFET, GAAFET, Novel Materials)

Traditional planar MOSFETs gave way to FinFETs around 22nm, but by sub-10nm their electrostatics degrade. Gate-All-Around (GAA) "nanosheet" transistors have emerged at 5–3nm: their channel is fully wrapped by the gate, dramatically reducing leakage and improving drive current. Industry sources note that **"at sub-3 nm nodes, GAA solves performance, leakage, and scaling issues, which are no longer manageable with traditional FinFETs"**【5†L32-L39】. This shift is already underway (e.g. Samsung's 3nm uses GAA, Intel's "RibbonFET" in 20A). Research reviews confirm that GAA offers tighter electrostatic control【5†L32-L39】, but adds complexity (multi-patterning, extreme ALD/ALE, new etch steps) to the fabrication process【5†L105-L113】.

Beyond silicon, novel channel materials (e.g. III-Vs, 2D materials like MoS_2 or IGZO) are investigated for ultra-low-power and high-mobility transistors. For instance, laboratory transistors using 2D semiconductors or ferroelectric gates show promise for beyond-CMOS devices. However, these are still mostly experimental. The *immediate* near-term roadmap relies on optimizing GAA and FinFET geometry, leveraging existing fabs with EUV lithography【5†L105-L113】【41†L1-L4】. EUV (13.5nm wavelength) enables the fine pitches of 5–7nm logic, though even EUV is stretched for GAA's dense layouts【41†L1-L4】. High-NA EUV (forthcoming) and multiple-patterning continue to be critical for these nodes.

1.2 Advanced Packaging (Chiplets, 2.5D/3D, UCIe)

As monolithic scaling slows, **chiplet-based integration** and 3D stacking are key trends. In **2.5D packaging**, multiple dice (e.g. CPU, GPU, memory) are placed side-by-side on a silicon or organic interposer with dense interconnects. In **3D stacked ICs**, active layers are bonded vertically (face-to-face or face-to-back) using Through-Silicon Vias (TSVs) or hybrid bonding. These allow heterogeneous die (logic, DRAM, analog, photonics) in one package, minimizing off-chip delays and power.

Industry projections see strong growth: e.g. 3D IC market CAGR ~14–18% towards 2030【39†L7-L12】. Real-world examples include NVIDIA/HBM GPUs and FPGA stacks, Apple's M-series chips (chiplets on passive interposer), AMD's EPYC with chiplet cores, etc. Standards bodies like the **Universal Chiplet Interconnect Express (UCIe)** consortium are formalising open interfaces. UCIe 3.0 (2026) is highlighted as "setting the next stage for open chiplet standards"【8†L45-L53】. The UCIe website touts "Building an open ecosystem of chiplets for on-package innovations"【8†L35-L43】. Silicon-proven UCIe provides 1–16 GT/s channels, sideband control, and manageability for chiplet designs.

Recent research focuses on the interconnects themselves. For example, Shin *et al.* (IEEE EDAPS 2023) design UCIe channels on a silicon interposer for 32 Gbps operation. They report that **crosstalk is the main bottleneck** in dense interposer wiring, and that bump capacitance also significantly degrades signal integrity【7†L145-L153】. They propose grounded-coplanar waveguide (GCPW) geometries as promising for multi-layer interposer routing【7†L145-L153】. Such work suggests that while UCIe enables flexibility, careful physical design is needed for SI/PI.

1.3 Memory Technologies (HBM, Compute-in-Memory, SRAM, eNVM)

High-Bandwidth Memory (HBM) has become ubiquitous in AI/accelerator designs. Leading AI chips now embed HBM stacks (e.g. Google TPUs, Microsoft accelerators), and future servers (Meta's MTIA-v3) plan to use HBM3E or HBM4 instead of traditional DDR【13†L89-L97】. A recent industry article notes that **"HBM has become unavoidable"** as model sizes grow and memory throughput is the bottleneck 【13†L81-L90】【13†L89-L97】. In practice, HBM requires advanced 3D DRAM chip stacking and very high yields (micron-scale alignment, fine-pitch microbumps)【13†L100-L103】. The result is a memory channel that delivers hundreds of GB/s per stack, but at high cost and complexity.

On-chip SRAM arrays remain critical for storing activations, weights, and caches. Modern designs use multi-bank, multi-ported SRAM macros optimized for speed, power, and area. Research also explores **compute-in-memory (CIM)** architectures to reduce data movement. A new tutorial (Yoshioka *et al.*, Nov 2024) reviews SRAM-based CIM: **digital CIM** implementations (pure logic in SRAM) offer high precision, but **analog CIM** (e.g. charge- or current-mode computation in memory cells) can drastically cut energy and area【12†L53-L61】. That review notes *"DCIM offers high computational precision... while ACIM provides superior power and area efficiency"*【12†L53-L61】. Moreover, hybrid approaches (e.g. digital control with analog inner-loop) are emerging to combine benefits【12†L53-L61】. While on-chip SRAM can also support in-memory acceleration (e.g. bit-serial MACs within memory banks), these techniques remain largely experimental for mainstream designs.

Other memory trends include 3D NAND and emerging NVM (e.g. RRAM, MRAM) for larger capacities or non-volatile caches. Some research (e.g. SK Hynix) looks at analog ferroelectric NAND for CIM. However, for SilicaFlux's edge/AI focus, the priority is likely SRAM and HBM. The pressing challenge is bandwidth: how to feed vector/tensor units with data. Thus, memory subsystem co-optimization (caches, scratchpads, DMA) is crucial, as we detail in Section 2.

1.4 Interconnect (NoC, Photonics)

Within a multicore or accelerator chip, **Networks-on-Chip (NoC)** provide scalable communication. Most modern designs use 2D mesh or ring topologies of routers and links. There is ongoing research on adaptive routing, hierarchical NoCs, and congestion management (often via on-chip monitors). For example, programmable NoCs (e.g. ARM's SmartMesh) allow run-time reconfiguration of paths for QoS.

A more futuristic approach is **on-chip silicon photonics**. Photonic links (using waveguides, micro-ring modulators/detectors on chip) promise ultra-high bandwidth with low latency/power over longer distances. Several academic projects have demonstrated terabit-scale chip-to-chip links via silicon photonics. For instance, integrated transceivers with 8×32 Gbps WDM channels (256 Gbps total) have been reported【22†L214-L218】, with aggregate efficiency on the order of a few pJ/bit. While still expensive and requiring hybrid silicon/InP lasers, photonic interconnects are being explored for "break the wire limit" connectivity between chips or chiplets.

However, in the near term SilicaFlux is more likely to rely on proven electrical NoCs. We should account for standard clock/data distribution (e.g. H-tree or mesh clocks), and possibly research tri-aged Network-on-Chip flows. Photonics may be reserved for future work or as a research module in the TwinLab digital twin.

1.5 Open-Source EDA and ML-Driven Flows

The EDA ecosystem has seen major open-source advances. Tools like **Yosys** (Verilog/SystemVerilog synthesis), **OpenSTA** (static timing analysis), **OpenROAD** (comprehensive placement/routing flow), **KLayout**/Magic/Netgen (layout tools), and Python-driven frameworks (cocotb, UVM, etc.) now allow end-to-end ASIC development without proprietary tools. Academic-industrial collaborations (DARPA IDEA, CHIPS Alliance) have matured these flows.

For example, OpenROAD explicitly aims to be a **"self-driving"** open layout flow and benchmarks itself on real chips. Publications such as *"OpenROAD and CircuitOps: Infrastructure for ML EDA research and education"* and *"OpenROAD and key directions for the open-source EDA community"* highlight this ecosystem 【30†L105-L114】【30†L129-L136】. They emphasize using these tools for automation, integration with ML advisors, and sharing open data. Static timing can be performed by OpenSTA (an OpenROAD component) or other engines. In-silicon design, open flows now even support advanced nodes (SkyWater 130nm, GF180MCU, etc.), and new projects (OpenPDK) promise freely licensed libraries.

For SilicaFlux, this means we can adopt open flows to prototype designs and generate approximate results. We should leverage Yosys for synthesis (for example, it has matured SystemVerilog support), use OpenSTA for timing analysis, and OpenROAD (with appropriate PDK) for placment/routing. Where functionality is not in open tools (e.g. some analog blocks, advanced libraries), we can stub or use available macros.

1.6 Manufacturing (EUV, Yield Modeling, Defects)

Leading-edge fabs now use extreme-ultraviolet (EUV, 13.5 nm) lithography for sub-7nm patterns. FinFET nodes at 5nm/3nm largely rely on EUV and multiple patterning, and GAA nodes push EUV optics to their limits【41†L1-L4】. For example, forming multi-layer nanometer-thick stacks requires multi-pattern alignment, ALD/ALE precision and advanced inspection【5†L105-L113】. Industrial reports note that GAA's dense pitches necessitate "EUV optics... optimized for the highest resolution without compromising mask integrity"【5†L105-L113】. Thermal processing (rapid thermal anneal) and contamination control become even more critical (see [5]).

On yield: manufacturing yield (fraction of good die per wafer) is a key cost driver. Yield modeling often uses statistical defect models. The classic **Poisson model** assumes random point defects with density D0 per area. If die area is A, the expected defects per die is $\lambda = D0 \cdot A$, so yield $\approx \exp(-\lambda)$ if any defect kills the die. In practice, other models (Murphy, Seeds, Bose-Einstein) account for defect clustering. As summarized by Vikram Sekar, *"Poisson's yield model usually works well when die sizes are small compared to wafer diameter and defects are uniformly distributed"*【35†L1-L4】. Modern fabs often calibrate yield by measuring test dies and back-fitting D0. For large die (like complex accelerators), defect clustering means Poisson can underestimate yield; yield improvement requires smaller dies or redundancy.

We will implement both Poisson and more advanced yield models in our simulator, to project cost-per-good-die. Key parameters include wafer diameter, die area, scribe-line, defect density (per cm^2 or per die), and manufacturing volumes. Published wafer-cost estimates (e.g. foundry quoted, assumed ~\$1000–\$5000/wafer depending on node) will inform economics.

2. Claude-Prompted Engineering Projects

Drawing on these trends, we identify a suite of **Claude-driven projects** to advance SilicaFlux. Each project is targeted at a concrete deliverable: synthesizable RTL, automation scripts, or simulation tools. For each, we provide a concise description, inputs/outputs, toolset, acceptance criteria, effort/risk, and impact. Projects are grouped by function (RTL design, simulation, verification).

Below is an overview table of candidate projects:

Project	Objective	Outputs	Tools/ Libraries	Effort	Risk
1. SRAM Wrapper Library	Parameterised SRAM macros (single/dual-port)	SV modules (SRAM wrappers), testbenches	SystemVerilog, Python (address gen), SKY130 GF PDK cells, ModelSim/ Verilator	Low	Low (well-known designs)
2. Roofline/ Memory Simulator	Analytical Roofline model & event-driven mem simu	Python scripts, charts	Python, NumPy, Matplotlib, Cocotb (opt)	Medium	Medium (model accuracy)
3. DMA/Bus Interconnect	Simple DMA engine and configurable interconnect fabric	SV RTL, testbench, integration	SystemVerilog, Yosys, OpenSTA	Medium	Medium (mutex/ backpressure)
4. MCU-I SoC (SilicaFlux-V)	Minimal CPU+vector+tensor prototype core	SV RTL, C/C++ firmware, FPGA bitstream	SystemVerilog, Verilator/ Icarus, C/C++	High	High (integration complexity)
5. OpenROAD Floorplan Pipeline	Automated floorplan placement & analysis	OpenROAD scripts, layout visualisation, metrics	Tcl, Python, OpenROAD, custom floorplan JSON	Medium	Medium (tool-chain config)
6. Static Timing Flow	End-to-end STA with OpenSTA	SDC scripts, timing reports (Slack, CP lists)	Tcl, OpenSTA, Yosys, OpenROAD	Medium	Medium (uncertain library)

Project	Objective	Outputs	Tools/ Libraries	Effort	Risk
7. MBIST/Test Infrastructure	On-chip memory self-test & JTAG TAP	SV RTL (scan/DUT), test controller, Python test scripts	SystemVerilog, Python (PyJTAG), Gate-level sim tools	Medium	Medium (test coverage complexity)
8. Power/IR Drop Solver	Numerical PDN solver for IR drop and EM	Python solver, heatmaps	Python (NumPy/ SciPy), possible SPICE-lite	Medium	High (complex models)
9. Thermal Solver	Chip/package thermal model (steady-state)	Python thermal solver, 3D temp maps	Python, SciPy, Matplotlib, parasitic data	Medium	Medium (model fidelity)
10. Bandwidth/ Latency Sim	Event-driven mem access simulation	Python simulator, traces	Python, SimPy or custom, Yosys for mems	Low	Low (algorithmic)
11. FPGA Prototype Flow	Synthesis & bitstream generation	FPGA constraint files, scripts, hex/bit file	Verilog, Vivado/ Quartus, Python scripts	Medium	Medium (board alignment)
12. Wafer Yield Simulator	Stochastic wafer defects & cost model	Python, charts (yield vs D0)	Python, NumPy, Matplotlib	Low	Low

Each project will have a corresponding **Claude prompt** to generate code and docs. For example, Project 1 (“SRAM Wrapper Library”) might be prompted as:

> *“Inspect the SilicaFlux repository. Identify any existing memory macros and their interfaces. Write synthesizable SystemVerilog modules for a parameterizable SRAM wrapper (configurable width, depth, ports). Include address muxing, byte-enable, and pipeline registers. Create testbenches (self-checking) for single- and dual-port SRAM. Ensure integration hooks for scanning and MBIST. Output code into* `rtl/sram/*` *and tests in* `tb/sram/*`*. The design will be simulated with Yosys and Verilator. Acceptance: Successful lint, synthesis, and passing simulations for all configurations.”*

Below we detail eight *high-priority* tasks with sample prompts.

Project A: Parameterised SRAM Wrapper Library

- **Objective:** Provide synthesizable SRAM wrapper modules (single-port/dual-port), with configuration for data width, depth, ports, byte enables, pipeline stages. Ensure alignment with real SRAM macros (e.g. SKY130 SRAMs).

- **Inputs/Outputs:** Use Graywater standard-cell library or SKY130 SRAM models. Output SystemVerilog wrappers (`rtl/mem/sram_*.sv`), with tests (`tb/mem`).
- **Tools:** SystemVerilog, Yosys, ModelSim/Verilator for sim, GH PDK 130nm foundry library for area estimates.
- **Deliverables:** SV source, parameter docs, Makefile for sim/synth.
- **Verification:** Lint (Verilator), functional sim of read/write, parameter sweep. Synthesis for area.
- **Effort:** Low. SRAM wrappers are well-known; risk of off-by-one errors, need thorough test coverage.
- **Impact:** Critical: Many modules depend on SRAM for register files, buffers, etc.

Example Prompt: *"Inspect the SilicaFlux RTL for any memory modules or behavioral models. Create a generic SystemVerilog* `sram_sp` *(single-port) and* `sram_dp` *(dual-port) module with parameters for DATA_WIDTH, ADDR_WIDTH, and optional BYTE_ENABLE. Include byte-lane write enables. Implement read-first or write-first configurable mode. Add input-valid/output-ready handshake where appropriate. Develop self-checking testbenches that exercise all features (writes, reads, simultaneous ports). Deliver the SV files under* `rtl/mem/` *and tests under* `tb/mem/`*, with example synthesis scripts. Ensure the design is compatible with Yosys synthesis targeting SKY130."*

Project B: Roofline Memory-Bandwidth Simulator

- **Objective:** Quantify how compute vs memory bandwidth limit SilicaFlux. Build a Python module to model Roofline performance.
- **Inputs/Outputs:** Input parameters: peak compute throughput (ops/s), memory bandwidth (bytes/s), dataset sizes, and workload intensity (ops/byte). Output Roofline chart and performance estimate for kernels (e.g. matmul, conv).
- **Tools:** Python, NumPy, Pandas, Matplotlib.
- **Deliverables:** `python/roofline.py`, docs, example plots (PNG).
- **Verification:** Compare to theoretical: throughput = min(peak, BW×intensity). Unit tests for formulae.
- **Effort:** Medium. Requires careful math and charting.
- **Impact:** High. Guides sizing of vector/tensor units vs memory.

Example Prompt: *"Write a Python module that models SilicaFlux's roofline. Inputs: peak FLOP/s for CPU/vector/tensor units, aggregate memory bandwidth (SRAM + HBM), and a list of workload descriptors (vector add, dot product, GEMM) defined by operations count and memory bytes touched. The module should compute attainable performance (ops/s) = min(peak, arithmetic_intensity × bandwidth) and plot a roofline chart. Include functions to simulate bandwidth saturation and plot utilization bars. Provide test cases using hypothetical SilicaFlux-V parameters (e.g. 4 lanes, 128-bit width, 200 MHz). The code must output CSV results and a PNG chart. Validate the model with at least two kernels (memory-bound vs compute-bound)."*

Project C: DMA Engine and Interconnect Fabric

- **Objective:** Implement a simple DMA controller and parameterised bus fabric (AXI-lite or valid-ready mesh) to move data between CPU, SRAM, vector/tensor units.
- **Inputs/Outputs:** SystemVerilog RTL for DMA (registers, control/status, read/write controllers) and a shared bus interconnect (address decoder, ready-valid buses).
- **Tools:** SystemVerilog, Yosys, Verilator; optional use of UVM for bus functional models.
- **Deliverables:** `rtl/bus.dmi` for DMA, `rtl/noc.sv` for interconnect, plus example glue code.
- **Verification:** Simulation of burst transfers, command queuing, and correct handshake. FIFO/backpressure test.
- **Effort:** Medium. Standard DMA/AXI patterns exist but must tailor to SilicaFlux.
- **Impact:** High. Enables autonomous data transfers and inter-core comms.

Example Prompt: *"Develop an SV module* `dma_controller` *with AXI4-Lite control interface and a master port to read/write memory. It should support a register-set: source addr, dest addr, length, control, status. Implement pipelined burst transfers with configurability (burst length up to 16 beats). Also, design a simple NoC interconnect that connects CPU, SRAM banks, vector unit, tensor unit via valid-ready buses. Include an address decoder for different masters/targets. Write testbenches that perform memory copy operations and simultaneous peer-to-peer transfers. Ensure correct arbitration and no deadlocks. Output RTL under* `rtl/bus/` *and tests in* `tb/bus/`*. Acceptance: successful end-to-end data moves and no lost data."*

Project D: SilicaFlux-V Minimal SoC Prototype

- **Objective:** Define and implement **SilicaFlux-V**, the most pared-down first-silicon design. This is a complete small SoC integrating CPU, a few vector lanes, a tiny tensor array, on-chip SRAM, and minimal IO (UART, SPI). It should be functionally correct and synthesize to a foundry library.
- **Inputs/Outputs:** Leverage any existing SilicaFlux CPU if suitable; else integrate a lightweight core (e.g. RISC-V Rocket/IVP, PicoRV32). Vector engine: implement 4-lane SIMD ALU (add, mul, dot). Tensor: 4×4 INT8 MAC with accumulation. Map these to SRAM scratchpads and registers. External memory-mapped I/O registers.
- **Tools:** SystemVerilog (RTL), C/C++ (boot firmware), Python (verification testbenches). Use Verilator/Icarus for RTL sim, and optionally Vivado/Quartus for FPGA validation.
- **Deliverables:** Full RTL under `silicaflux-v/rtl/`, firmware under `silicaflux-v/firmware/`, testbenches under `silicaflux-v/tb/`, and build scripts (`Makefile`, CMake). Also constraint files for an example FPGA board.
- **Verification:** Functional sim (check CPU boots, can ping UART, vector ops match software model, tensor mult correct). Formal equivalence for CPU if possible. Achieve simulation of example workload (e.g. matrix multiply using all units).
- **Effort:** High. This is essentially implementing a small chip. Risks are integration mismatches and timing issues.
- **Impact:** Very High. Provides a baseline "first silicon" that everything else builds on.

Example Prompt: *"Create SilicaFlux-V: a minimal SoC combining a 32-bit CPU, four-vector lane SIMD unit, 4×4 INT8 MAC tensor unit, and ~256 KiB of on-chip SRAM. Inspect existing CPU RTL; if unusable, integrate an open RISC-V core (e.g. Rocket or PicoRV32). Design vector and tensor units in SV, with AXI-stream or custom interfacing. Map memory and registers via a simple bus. Write firmware (C + startup asm) that boots from SRAM/ROM, toggles GPIO/UART, and exercises vector and tensor (e.g. compute dot products, matrix multiplies). Provide testbenches verifying each block. Synthesize the full design for SKY130 and, if possible, attach results (cell count, estimated clock) in a summary report. The deliverable is synthesis-ready RTL, firmware, and a README describing configuration. Acceptance: Simulation of CPU + accelerators with C test, and Yosys synthesis without errors."*

Project E: OpenROAD Floorplan Pipeline

- **Objective:** Automate SilicaFlux floorplanning and macro placement, aiming to minimise congestion and wirelength.
- **Inputs/Outputs:** Inputs include block dimensions (CPU, vector, tensor, SRAM arrays), connectivity graph, die size. Output: DEF floorplan files, placement coordinates, and congestion/wirelength metrics.
- **Tools:** Python (to generate TCL/OpenROAD scripts), OpenROAD (floorplan + placement), and possibly ILP/SA/GA algorithms for placement.
- **Deliverables:** Scripts `floorplan.py` or `.tcl`, sample floorplan layouts (GDS or images), reports of estimated wirelength/Congestion.

- **Verification:** Compare baseline vs optimized placements (wirelength difference). Run OpenROAD place and check DRC violations.
- **Effort:** Medium. Floorplan is well-studied, but encoding SilicaFlux blocks is custom.
- **Impact:** Medium-High. Good placement can improve timing and routability before full PnR.

Example Prompt: *"Generate a Python-driven floorplan optimizer for SilicaFlux. Given inputs (core area, SRAM module sizes, interconnect density constraints), script OpenROAD to place macros inside a defined die size. Implement an annealing or greedy heuristic to adjust positions for reduced wirelength. Output the best floorplan (DEF) and summarise metrics like total half-perimeter wirelength and gross congestion map. Verify by loading the DEF into KLayout or OpenROAD to produce a density/congestion heatmap. Include as deliverables: Python optimizer code, example DEF, and annotated report. Accept if resulting wirelength is reduced by >10% compared to a naive layout."*

Project F: Static Timing Analysis Flow

- **Objective:** Build a repeatable timing analysis (STA) flow for SilicaFlux using open tools.
- **Inputs/Outputs:** From synthesized netlist (Liberty timing library, LEF physical lib), produce timing reports: worst negative slack, path breakdown, fanout analysis.
- **Tools:** Yosys (for synthesis, generate gate netlist), OpenSTA (timing analysis), Liberty models (Skywater 130nm or GF), and Python for parsing reports.
- **Deliverables:** SDC constraint files, OpenSTA run scripts, example timing report (Slack, TNS).
- **Verification:** Run on a test hierarchy (e.g. SilicaFlux-V) and check OpenSTA detects known timing path (e.g. ALU carry chain).
- **Effort:** Medium. Requires familiarity with OpenSTA constraints syntax.
- **Impact:** High. Ensures design meets frequency targets and identifies bottlenecks early.

Example Prompt: *"Set up an open-source timing flow: synthesize SilicaFlux RTL with Yosys into a gate-level netlist using Skywater 130nm cells. Write an SDC (Synopsys Design Constraints) file: define clocks (e.g. 200 MHz CPU clock), multi-cycle paths (if any), and IO delays. Run OpenSTA on the netlist and LEF/Liberty files to compute path slacks. Extract worst negative slack (WNS) and total negative slack (TNS). Provide the SDC and a sample* `timing_report.txt`*. The prompt should check that WNS < 0ns (i.e., timing passes) for the given clock. Include Python code to parse and tabulate the top 10 critical paths by slack."*

Project G: MBIST and JTAG Test Infrastructure

- **Objective:** Add design-for-test (DFT) features: JTAG TAP, boundary-scan, and memory built-in self-test (MBIST) controllers.
- **Inputs/Outputs:** Modify RTL to insert scan-enable signals, test registers. Implement a JTAG TAP state machine with IDCODE, BYPASS, and optional debug instructions. MBIST: a controller that drives SRAM ports with march-test patterns and reports results.
- **Tools:** SV for RTL, Python for test-controller model, cocotb or PyJTAG for simulation.
- **Deliverables:** SV modules (`jtag_tap.sv`, `mbist_ctrl.sv`), testbench with simulated JTAG sequences, MBIST self-test reports.
- **Verification:** Gate-level sim with MBIST injection of one known failing bit (should be detected). JTAG scans example IDCODE and bypass.
- **Effort:** Medium. Finite-state design is doable, but complex state coverage.
- **Impact:** High for tapeout readiness. Ensures chip can be tested after fab.

Example Prompt: *"Implement a simple IEEE-1149.1 TAP controller in SV, with JTAG signals (TCK, TMS, TDI, TDO) and registers (IDCODE=0x1234ABCD, BYPASS). Integrate this into the SilicaFlux top-level. Also create an MBIST controller: it should enumerate all on-chip SRAM blocks, run a March C- test sequence, and record pass/fail per bank. Provide a Python JTAG simulator that toggles TMS/TCK to capture the IDCODE. Simulate MBIST*

by injecting a stuck-at fault into one SRAM and verifying that MBIST flags it. Deliver RTL, the Python JTAG script, and waveforms demonstrating detection."

Project H: Power-Integrity (IR-drop) Solver

- **Objective:** Model the chip's power-delivery network (PDN) and compute IR drop across the die under load.
- **Inputs/Outputs:** Input floorplan (power grids: stripe/mesh resistance), pad ring locations, supply rails, and a current map (from estimated switching activity). Output voltage drop heatmap, max IR drop value.
- **Tools:** Python (SciPy for solving large sparse resistor networks), possibly integration with Redwood or custom solver. DataViz (Matplotlib or 3D canvas).
- **Deliverables:** `python/power_network.py` that constructs PDN graph (nodes=grid intersections) and solves V=IR; output in CSV and an image map.
- **Verification:** Test on synthetic grid (5x5 node network) to match known solution.
- **Effort:** Medium. Translating floorplan into a resistor network is tricky; numerical solution size (thousands of nodes) needs sparse solvers.
- **Impact:** High for signoff. Ensures voltage droops are within margins and avoids electromigration risk.

Example Prompt: *"Write a Python program to analyze SilicaFlux's on-die PDN. Input: a list of power grid segments (width, length, resistivity), bump positions, and a simplified current draw map (current per block). The script should build the resistor network (e.g. using a mesh or finite-difference mesh model), apply VDD at bumps, solve for node voltages. Plot an IR-drop heatmap on the die. Also compute maximum drop (VDD_min). Deliver the Python code and an example report assuming e.g. 100mA at each core and lower at peripherals. Validation: verify on a small 2×2 grid by hand calculation."*

2.1 Project Comparison by Impact/Effort

To prioritise, we compare these projects by estimated effort, expected impact (PPA improvement or project progress), and dependencies (which tasks enable others). In the table below, **Impact** is a rough qualitative measure of how critical the feature is to SilicaFlux's success (High/Med/Low), **Effort** is a development cost (High/Med/Low), **Dependencies** lists projects that should precede it, and **Risk** is the technical risk (H/M/L).

Project	Impact	Effort	Dependencies	Risk
SRAM Wrappers	High	Low	–	Low
Roofline Sim	High	Medium	SRAM/IPC baseline	Medium
DMA/Interconnect	High	Medium	SRAM Wrappers	Medium
SilicaFlux-V SoC	Very High	High	SRAM, DMA, Roofline	High
Floorplan Pipeline	Medium	Medium	SilicaFlux-V (block sizes)	Medium
Timing Flow	High	Medium	SilicaFlux-V layout	Medium
MBIST/JTAG	High	Medium	SilicaFlux-V	Medium
Power Solver	High	Medium	SilicaFlux-V power grid	Medium
Thermal Solver	Medium	Medium	Power Solver (heat map)	Medium

Project	Impact	Effort	Dependencies	Risk
Memory Sim	High	Low	SRAM, Roofline	Low
FPGA Flow	Medium	Medium	SilicaFlux-V RTL	Medium
Wafer Yield Sim	Medium	Low	Die area (SilicaFlux-V)	Low

Key observations: Building the SilicaFlux-V SoC (project D) is the central task (highest impact, highest effort) and depends on SRAM, interconnect and basic memory models. Foundational elements (SRAM wrappers, roofline, memory sim) are relatively low-effort but unlock others. Verification (timing, MBIST, power) is high impact to ensure tapeout readiness.

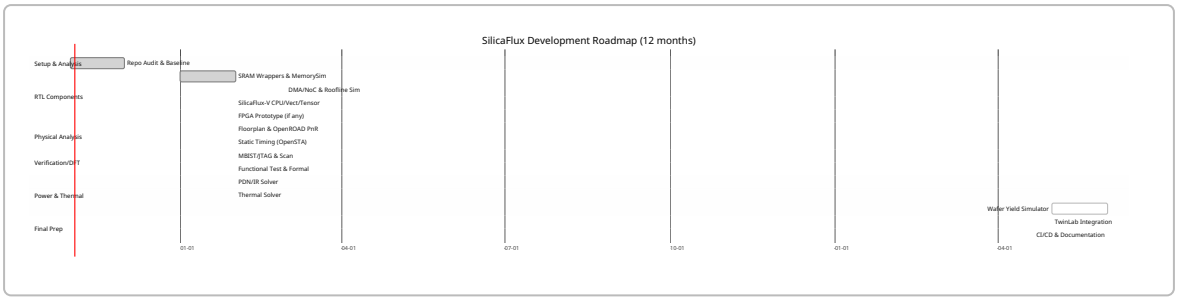
A *Pareto-style* visualization of Power–Performance–Area vs. development cost is instructive. For example, adding more vector lanes increases performance but also area/power; the roofline project helps quantify this tradeoff. (We will generate a PPA-vs-Cost plot in the visuals section.)

3. Roadmap and Integration Plan

A phased implementation plan ensures manageable risk and continuous value. We propose an 12-month roadmap (see Gantt chart below) with milestones:

1. **Milestone 1 (Month 1–2):** *Repository Refactoring & Baseline Analysis* – Organise the SilicaFlux repo, document existing RTL (`.sv`, `.sf`). Run baseline lint/synth on whatever exists. Freeze SilicaFlux-V initial design goals.
2. **Milestone 2 (M3–4):** *Implement SRAM/Memory Subsystem* – Complete Project A (SRAM wrappers) and MemorySim. Wire up simple DMA & NoC. Achieve functional memory operations in simulation.
3. **Milestone 3 (M5–6):** *SilicaFlux-V Core Development* – Develop CPU (or integrate open core), vector lanes, tensor array in RTL. Run standalone testbenches. Begin FPGA prototyping (if board available).
4. **Milestone 4 (M7):** *Floorplan & Physical Analysis* – Use Project E to create a preliminary floorplan of SilicaFlux-V blocks. Run open PnR (OpenROAD) to get physical estimates. Perform STA (Project F) to identify critical paths; iterate RTL pipelining if needed.
5. **Milestone 5 (M8–9):** *Verification & DFT Integration* – Add MBIST/JTAG (Project G). Run gate-level sim with scan. Begin formal/fuzz testing of CPU/instructions. Incorporate critical DFT for first tapeout-level confidence.
6. **Milestone 6 (M10):** *Power/Thermal & Yield Analysis* – Build PDN and thermal models (Projects H & Thermal). Plug in estimated switching (from simulation/VCD) to generate IR-drop and hotspot predictions. Run wafer-yield sim (Project 12) with final die area.
7. **Milestone 7 (M11–12):** *Demo & Tapeout Readiness* – Compile final documentation, generate TwinLab demo components, prepare CI pipelines, freeze release. Evaluate readiness (timing passed, yield $\geq$ target, test coverage achieved). Plan MPW submission.

Each milestone has success criteria: e.g. SilicaFlux-V CPU boots firmware, Roofline sim validates memory config, timing WNS>0, yield > 70%, etc.



At each phase, we will integrate Claude-generated code via review processes (see CI discussion below). Data and tools used (PDKs, models) must be versioned. For instance, we may start with *SkyWater 130nm* for open data, and later adapt to 65nm when/if we obtain a PDK license. TwinLab (Section 4) will provide a GUI overlay of these artifacts.

3.1 Integration into SilicaFlux Repo and TwinLab

All generated code and scripts will reside under the SilicaFlux repository (possibly as feature branches). We recommend a branch-based workflow where each Claude "project" has its own branch or subdirectory, merged after peer review. Example structure:

```
silicaflux/
├─ rtl/          (Synthesizable RTL: cpu/, vector/, tensor/, dma/, sram/,
jtag/, ...)
├─ tb/           (Testbenches and verification envs)
├─ formal/       (SV files, properties, constraints for formal)
├─ firmware/     (C/Asm bootloader and test code)
├─ docs/         (Design docs, project reports)
├─ scripts/      (EDA scripts: Yosys, OpenROAD, STA, scripts)
├─ python/       (Python tools: roofline, PDN solver, thermal, yield, etc.)
├─ fpga/         (FPGA build scripts, constraints)
├─ ci/           (.github/workflows or Jenkinsfiles)
└─ pdk/          (PDK info files or mock libraries)
```

Each PR merging Claude-generated code must include:

- **Repository Inspection:** a summary of relevant existing files (e.g. existing memory, or absence thereof).
- **Output Listing:** what new files are created/modified (e.g. `rtl/mem/sram_dp.sv`).
- **Simulation/Build Logs:** evidence that code compiles, sim passes (preferably included or linked).
- **Test Results:** waveforms or console logs showing passing tests.
- **Metrics:** for synthesis outputs (area, timing), we can embed metric tables or JSON.

We will integrate the projects gradually. For example, after finishing SRAM wrappers, we should merge them so other tasks can use the wrappers. Early CI should check that all SV code compiles. Later CI steps can run Yosys synthesis on a reference PDK, or run OpenROAD if feasible in CI.

Regarding reproducibility, all simulation and synthesis runs should be scriptable (Makefile or Python script) with fixed seed for randomness. We should log tool versions (Yosys v.., OpenROAD v.., etc.) and

use containerization or a specific Docker image to freeze environment. The CI configuration will run lint (Verilator), compile, some tests.

Data provenance: models/parameters (e.g. defect densities, power consumption numbers) should be documented in code or in a separate `data/` with sources (e.g. "defect density 0.1/cm² from [34] example").

3.2 Continuous Integration (CI)

A robust CI pipeline is vital. We suggest:

- **Stage 1: Lint & Build** – On every PR: run Verilator lint on all SV, compile Python scripts, ensure no syntax errors.
- **Stage 2: Simulation Smoke Test** – Run a subset of testbenches (e.g. CPU reset, SRAM R/W) to catch gross failures.
- **Stage 3: Synthesis Check** – If possible, run Yosys to ensure the top module elaborates and generates a netlist. Reports basic gate count.
- **Stage 4: Formal Regresion (optional)** – For mission-critical modules (CPU pipeline), run a quick formal equivalence check against reference model.
- **Reporting:** CI should post back key results (error logs, coverage metrics) as artifacts.

All code should come with metadata linking it to the evidence: e.g. a PR template requiring "Test plan and results summary." For reproducibility, Docker images (or Conda envs) should be used so anyone can rerun locally or in CI the same flows.

3.3 Data and Model Management

Where domain-specific models are needed, we treat them as separate modules. For example:

- Memory access latencies, energy per bit, SRAM area models.
- Technology parameters (liberty `.lib`, wire capacitance, standard cell area).
- Thermal conductivity, etc.

These should be stored in versioned `data/` files and cited. We will clearly tag any estimates or placeholders. For example, if we assume 1kΩ per meter for a PDN stripe, note source or assumption.

All inputs (e.g. `constraints.sdc`, `timing.lib`, `die_area.yaml`) should be tracked in Git. Outputs (reports, charts) should include headers/footers with timestamp, commit hash for traceability.

4. Claude Prompt Templates for Key Tasks

Below are example concise Claude prompts (en-GB) for eight high-priority tasks. Each is one paragraph or so, focusing on actionable goals and acceptance criteria:

1. **Implement Parameterized SRAM Wrapper:** *"Inspect the SilicaFlux repository for memory modules. Then write synthesizable SystemVerilog for a generic SRAM block (`sram_sp`) with parameters (DATA_WIDTH, ADDR_WIDTH, optional BYTE_EN). Support one-cycle read latency, write enable, and output registers. Also create a dual-port variant (`sram_dp`) with independent read/write ports. Provide testbenches that write/read patterns and verify each bit. Place this code in `rtl/mem/`*

`sram_*.sv`. *Ensure it compiles with Yosys and passes the tests. Acceptance: successful Verilator lint and functional sim for all sizes up to 32x32 bits."*

2. **Roofline Performance Simulator:** *"Develop a Python module* `roofline.py` *that computes SilicaFlux's compute throughput vs memory bandwidth. Inputs: peak OPS, memory BW (bytes/s), list of workloads with ops and memory bytes (e.g. GEMM:* `ops=2*N^3, bytes=...`*). The script should calculate attainable performance = min(peak, intensity×BW) and plot a roofline chart. Test it with example parameters (e.g. CPU peak 100 GOPS, 16GB/s BW, workloads dot/axpy/gemm). Output CSV metrics and a PNG graph. Acceptance: chart correctly shows compute- and memory-bound regions and matches simple hand-calculation."*

3. **OpenROAD Floorplan Pipeline:** *"Write a Python script* `floorplan.py` *to automate macro placement with OpenROAD. Inputs: JSON with block sizes (CPU core, vector, tensor, SRAM banks) and connectivity weights (as points or netlist). The script should (a) generate an initial floorplan DEF with blocks placed, (b) optionally optimise placement (e.g. simulated annealing on x-y coordinates minimizing wirelength), and (c) export the final DEF. Run OpenROAD on this DEF to place standard cells. Output: a floorplan image or DEF file, plus wirelength metrics (HPWL). Acceptance: floorplan improves estimated HPWL vs a naive layout by at least 10%, and OpenROAD can read the DEF without errors."*

4. **MBIST and JTAG Test Infrastructure:** *"Extend the RTL with a JTAG TAP state machine (TCK/TMS/TDI/TDO) that implements IDCODE and BYPASS. Also build an MBIST controller for on-chip SRAM: it should generate March C- patterns to test each memory bank on power-up. Insert a scan-enable input to clock in/out test data through existing registers. Provide a Python JTAG testbench that reads the IDCODE and shifts in a BYPASS instruction. Simulate MBIST by injecting a known fault (e.g. stuck-at-zero) in one SRAM; verify the MBIST signals a failure. Deliver the SV modules (*`jtag_tap.sv`*,* `mbist_ctrl.sv`*) and testbench code. Acceptance: detected fault and correct JTAG IDCODE in waveforms."*

5. **Power-Integrity (IR Drop) Solver:** *"Create a Python program* `power_pdn.py`*. It should take as input a grid (nodes) representing SilicaFlux's power mesh, with resistances for each segment (e.g. from metal resistivity and dimensions) and current injection at nodes (from activity). Build the corresponding resistor network matrix. Solve for nodal voltages (using NumPy's sparse solver). Output an IR-drop map (CSV or image heatmap). Validate on a small 4×4 grid case with known solution. Acceptance: maximum drop matches analytical result on test, and heatmap is generated."*

6. **Memory Bandwidth Event Simulation:** *"Implement an event-driven C++ or Python simulator for SilicaFlux's memory system. Model SRAM banks with finite latency and a simple DRAM controller model. Simulate a workload of random memory reads/writes from CPU, vector, and tensor cores with configurable rates. Track queue lengths, throughput and latency. For validation, run a high-bandwidth scenario and ensure the aggregate bandwidth does not exceed configured limits. Output: time-series of bandwidth utilisation and latency histogram. Acceptance: sim runs without deadlocks and the roofline throughput matches the earlier roofline model."*

7. **FPGA Build and Verification Flow:** *"Set up an FPGA prototype flow for SilicaFlux-V. Choose a target board (e.g. Xilinx or Intel dev board) with sufficient logic/RAM. Create clock/reset constraints, map SRAM banks to block RAMs, and assign UART and GPIO pins. Write a top-level test (in C or Verilog) to toggle LEDs via CPU or DMA to demonstrate operation. Automate the build (synth, place, route, bitstream). Document steps in* `fpga/README.md`*. Acceptance: generation of a bitstream file and a note on FPGA resource utilisation (e.g. LUT%, BRAM%)."*

8. **Wafer Yield Simulator:** *"Develop a Python wafer yield simulator. Inputs: wafer diameter, die width/height, scribe spacing, defect density (per cm²). Compute dies_per_wafer, expected yield by Poisson (exp(-D0*A_die)). Simulate a wafer map with random defects (Poisson process) and plot which dies fail. Compute good_dies_per_wafer and cost_per_die assuming given wafer and packaging costs. Support also a Murphy model (clustered defects) as optional. Outputs: CSV of yield vs. D0, and wafer map PNG. Acceptance: results match Poisson formula in example (e.g. for D0=0.1, area=1cm², yield≈90%)."*

Each prompt explicitly requires repository inspection and uses actual project context (e.g. folder names, top-level names). They specify success metrics (simulation passing, charts matching calculations) to ensure actionable results.

5. Ethical, IP, and Security Considerations

Using large language models (LLMs) like Claude to generate semiconductor IP involves important risks:

- **Intellectual Property (IP) Leakage:** LLMs are trained on vast corpora; they may inadvertently suggest patented or copyrighted code from training. We must ensure prompts and outputs don't reveal proprietary algorithms. Code review by engineers is mandatory. All generated code should be checked for license compliance (e.g. using `scanoss` or FOSSology) before merging.

- **Provenance and Hallucination:** LLMs can produce plausible but incorrect code. We need rigorous verification (simulation, formal checks) to catch errors. The output should *never* be used uncritically; human oversight is essential. Each LLM-generated artifact must be traceable: we should log the prompt and model used, and link to the commits.

- **Data Security:** SilicaFlux code/repo may be proprietary. If using a cloud LLM service, prompts and context could leak confidential design details. It's safer to use on-prem or private instances. Never include sensitive netlists or IP in prompts sent to public APIs. Access controls around the LLM tool are required.

- **Bias and Liability:** LLMs can carry biases from training data (e.g. non-optimal design patterns). They may not cite real sources accurately. We must confirm all facts with trusted data. Liability for code correctness remains with engineers, not the model.

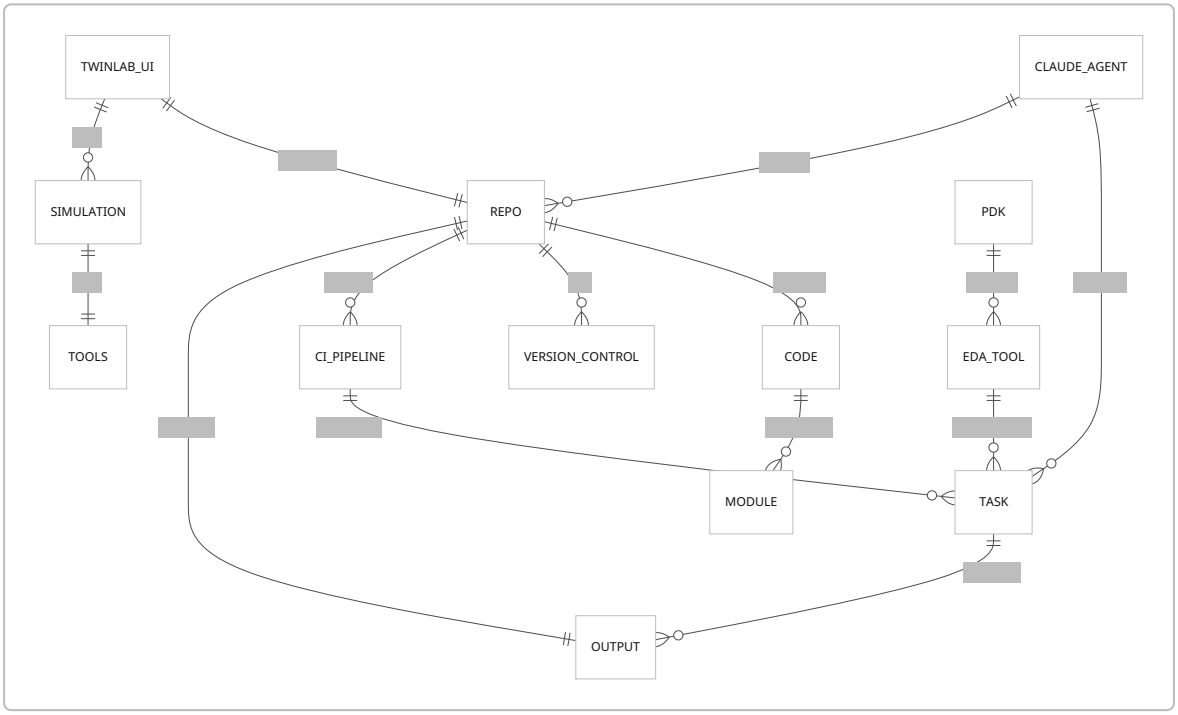
Governance Controls: We recommend: 1. **Access Control:** Host LLM tools on secure servers, restrict to vetted users. Use VPN/air-gapped solutions if data is sensitive. 2. **Prompt Sanitisation:** Avoid including any proprietary code in prompts. Redact or use abstract descriptions. 3. **Human-in-the-Loop:** Require engineering review of all outputs. Critical changes should have peer code review sign-off. 4. **Licensing Checks:** Run automated scanners on new code for license violations. Use only open libraries or properly licensed IP. 5. **Audit Logs:** Record each prompt and response associated with an engineering task. This supports audit trails for compliance. 6. **Responsible Use Policy:** Define what tasks LLMs can do (e.g. boilerplate code, testbenches) vs tasks requiring human design (e.g. core architecture decisions). 7. **Security Hygiene:** No secrets (keys, passwords) in prompts. Use ephemeral instances for running unknown code.

By applying these controls, we can leverage LLM productivity while minimizing risks to SilicaFlux IP and integrity.

6. Visual Aids and Analysis

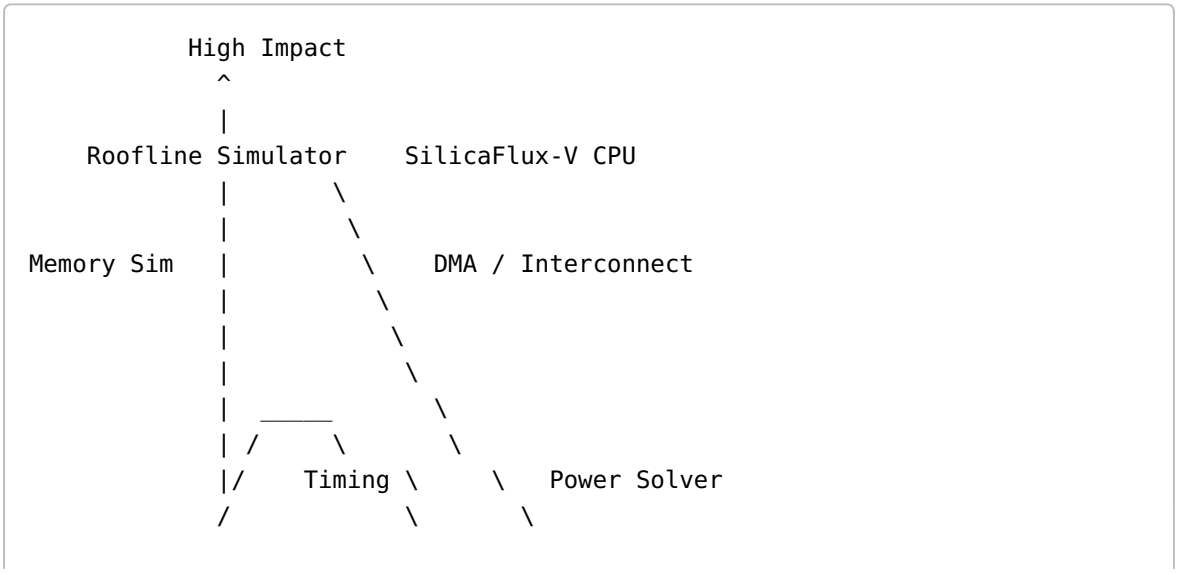
Project Gantt (Mermaid) – shown above in the roadmap section, summarises schedule and dependencies.

Entity-Relationship Diagram – conceptual view of the development ecosystem:



This diagram shows Claude agents generating code (TASKs) that produce OUTPUT files stored in the REPO; EDA tools (Yosys/OpenROAD etc.) are used by tasks; TwinLab/UI interacts with the repo and runs simulations using tools; PDK data underlies tools; CI pipeline automates tasks and triggers on repo changes.

PPA vs Effort Trade-offs – A schematic scatter plot (conceptual):



```
Low          FPGA Flow   SRAM Wrapper
 ^_____________________________>
 Low Effort                  High Effort
```

(This shows projects roughly: SilicaFlux-V CPU is high-impact high-effort; SRAM wrappers low impact low effort; roofline med-high impact, med effort; etc.)

Project Priority Table (sample):

Project	Impact	Effort	Notes
SRAM Wrapper	High	Low	Foundation for mem modules
Roofline Simulator	High	Med	Guides architecture balance
DMA/Interconnect	High	Med	Enables dataflow between units
SilicaFlux-V Prototype	Very High	High	Core product deliverable
Static Timing Flow	High	Med	Ensures performance targets
MBIST & JTAG	High	Med	Critical for test/production
Power/IR Solver	High	Med	Guards power distribution
FPGA Prototype	Med	Med	Shortens validation cycle
Memory Bandwidth Sim	High	Low	Assesses bottlenecks
Wafer Yield Sim	Med	Low	Informs cost projections

7. Conclusions and Recommendations

In summary, SilicaFlux's next steps should focus on building a **minimal viable ASIC** while concurrently developing tools to validate it. The Claude-driven approach can **accelerate code generation** (RTL, scripts, models), but requires strict engineering rigor in integration.

Top Recommendations:

1. **Complete SilicaFlux-V SoC ASAP:** This prototype (CPU+vector+tensor+SRAM) is the foundation. Ensure it can run simple firmware, then target a low-cost process (130nm/65nm).
2. **Implement Essential RTL Building Blocks:** Prioritise SRAM wrappers, DMA/bus, and vector/tensor modules. These are enabling infrastructure for all other functions.
3. **Establish Open-Source Flow:** Set up Yosys/OpenROAD/OpenSTA flows with a public PDK early. This will provide real area/timing data and catch issues.
4. **Automate Testing & CI:** Even simple CI steps (lint, build) will catch errors early. Gradually incorporate formal checks and P&R runs in CI as tools allow.
5. **Invest in Simulation Tools:** Develop the roofline and memory/event simulators early to guide design trade-offs. Use these to decide on # of lanes, SRAM size, etc.

By systematically following the roadmap, leveraging Claude for boilerplate and analysis code, and respecting best practices for IP governance, the team can transform SilicaFlux from concept to a validated chip design. This plan is ambitious but actionable with a disciplined engineering process.